1.2. Tại sao kiểm thử lại cần thiết?

Tổng quan

```
KIỂM THỬ = Hình thức KIỂM SOÁT CHẤT LƯỢNG

Mục đích: Đạt được mục tiêu trong giới hạn:
├─ 📏 Phạm vi (Scope)
├─ ⏰ Thời gian (Time)
├─ 💎 Chất lượng (Quality)
└─ 💰 Ngân sách (Budget)
```

Ai có thể đóng góp vào Testing?

```
┌──────────────────────────────────────────────────────────┐
│  Sự đóng góp của kiểm thử ≠ Chỉ của Testing Team         │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  BẤT KỲ STAKEHOLDER NÀO cũng có thể sử dụng              │
│  kỹ năng testing để đưa dự án đến thành công             │
│                                                          │
│  • Developers: Unit testing, code review                 │
│  • Business Analysts: Requirements review                │
│  • Product Owners: Acceptance testing                    │
│  • End Users: UAT, beta testing                          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.2.1. Đóng góp của kiểm thử đối với thành công

Các đóng góp chính

```
        KIỂM THỬ ĐÓNG GÓP VÀO THÀNH CÔNG
                    │
        ┌───────────┼───────────┬───────────┐
        ↓           ↓           ↓           ↓
   PHÁT HIỆN    ĐÁNH GIÁ    ĐẠI DIỆN    TUÂN THỦ
   DEFECTS      CHẤT LƯỢNG  NGƯỜI DÙNG  QUY ĐỊNH
```

#	Đóng góp	Chi tiết	Giá trị
1️⃣	**Phát hiện defects hiệu quả về chi phí**	• Testing tìm ra defects • Developer debugging để fix • → Nâng cao chất lượng gián tiếp	Tiết kiệm chi phí (phát hiện sớm = fix rẻ)
2️⃣	**Đánh giá trực tiếp chất lượng**	• Đo lường chất lượng tại mỗi giai đoạn SDLC	Quản lý dự án tốt hơn, quyết định sáng suốt

		• Hỗ trợ quyết định: chuyển giai đoạn, release	
3️⃣	**Đại diện cho người dùng**	• Tester đại diện cho nhu cầu user • Thay thế việc mời user tham gia (tốn kém)	Đảm bảo sản phẩm đáp ứng nhu cầu thực tế
4️⃣	**Đáp ứng yêu cầu tuân thủ**	• Hợp đồng • Pháp luật • Tiêu chuẩn quy định	Tránh rủi ro pháp lý, đảm bảo compliance

Sơ đồ chi tiết: Testing đóng góp như thế nào?

```
┌─────────────────────────────────────────────────────────┐
│  1. PHÁT HIỆN DEFECTS HIỆU QUẢ VỀ CHI PHÍ               │
└─────────────────────────────────────────────────────────┘

Testing → Tìm defects → Debugging (fix) → Chất lượng ↑

Chi phí:
└─ Phát hiện sớm (requirements): $1
└─ Phát hiện muộn (production): $100
   → Testing tiết kiệm 99x chi phí!

┌─────────────────────────────────────────────────────────┐
│  2. ĐÁNH GIÁ TRỰC TIẾP CHẤT LƯỢNG                       │
└─────────────────────────────────────────────────────────┘

Tại mỗi giai đoạn SDLC:
├─ Requirements → Static testing
├─ Design → Design review
├─ Development → Unit testing
├─ Integration → Integration testing
└─ System → System testing

Metrics: Test coverage, pass rate, defect density
         ↓
   Quyết định: "Có release được không?"

┌─────────────────────────────────────────────────────────┐
│  3. ĐẠI DIỆN CHO NGƯỜI DÙNG                             │
└─────────────────────────────────────────────────────────┘

Option A (Không khả thi):
└─ Mời nhóm users tham gia dự án
   → ❌ Chi phí cao, khó tìm users phù hợp

Option B (Khả thi):
└─ Testers đại diện cho users
   → ✅ Hiểu nhu cầu users, test theo user perspective
   → ✅ Đảm bảo sản phẩm đáp ứng nhu cầu thực tế
```

```
┌──────────────────────────────────────────────────────────┐
│  4. TUÂN THỦ YÊU CẦU                                     │
└──────────────────────────────────────────────────────────┘

Testing được yêu cầu bởi:
├─ 📜 Hợp đồng: "Phải test 95% code coverage"
├─ ⚖️ Pháp luật: GDPR, HIPAA, SOX compliance
└─ 📋 Tiêu chuẩn: ISO 25010, FDA regulations
```

1.2.2. Kiểm thử và Đảm bảo chất lượng (QA)

Sự khác biệt cơ bản

```
TESTING ≠ QA (Quality Assurance)

Mọi người thường nhầm lẫn, nhưng chúng KHÁC NHAU!
```

	TESTING	QA (Quality Assurance)
Định nghĩa	Kiểm soát chất lượng	Đảm bảo chất lượng
Tiếp cận	🎯 **Hướng sản phẩm** (Product-oriented)	🔄 **Hướng quy trình** (Process-oriented)
Tính chất	🔧 **Khắc phục** (Corrective)	🛡️ **Phòng ngừa** (Preventive)
Tập trung vào	Tìm và sửa defects trong sản phẩm	Cải tiến quy trình để tránh defects
Nguyên tắc	"Tìm lỗi để sửa"	"Quy trình tốt → Sản phẩm tốt"
Vai trò	Chủ yếu: Testing team	**Trách nhiệm của TẤT CẢ MỌI NGƯỜI**
Áp dụng cho	Sản phẩm (code, documents)	Quy trình (dev process, test process)
Khi nào	Trong và sau development	Xuyên suốt SDLC

Sơ đồ so sánh trực quan

```
╔═══════════════════════════════════════════════════════════╗
║                    TESTING vs QA                          ║
╚═══════════════════════════════════════════════════════════╝
╔                                                           ╗
║  TESTING (Kiểm soát chất lượng)                           ║
║  ──────────────────────────────                           ║
║  Hướng sản phẩm → Tìm lỗi trong sản phẩm                  ║
║                                                           ║
║  [Sản phẩm] → TEST → Tìm defects → FIX → [Sản phẩm tốt hơn]  ║
║                                                           ║
║  ═══════════════════════════════════════════════           ║
║                                                           ║
║  QA (Đảm bảo chất lượng)                                  ║
║  ──────────────────────                                   ║
```

```
╟  Hướng quy trình → Ngăn ngừa lỗi bằng quy trình tốt           ╟
╟                                                                ╟
╟  [Quy trình tốt] → THEO ĐÚNG → [Sản phẩm tốt]                ╟
╟                                                                ╟
╚═══════════════════════════════════════════════════════════════╝
```

Vai trò của Testing results trong QA và Testing

```
┌──────────────────┐
│ TESTING RESULTS  │
│  (Kết quả test)  │
└──────────────────┘
         │
         │
    ┌────┴────┐
    ↓         ↓

┌─────────┐  ┌──────────────┐
│TESTING │  │     QA      │
└─────────┘  └──────────────┘

    │            │
    ↓            ↓
Sửa lỗi    Cải tiến quy trình
(Fix bugs) (Improve process)
```

Sử dụng bởi	Mục đích	Ví dụ
Testing	Sửa lỗi trong sản phẩm	Tìm thấy 50 bugs → Fix 50 bugs
QA	Phản hồi về hiệu quả quy trình	"Tại sao có 50 bugs? → Quy trình code review chưa tốt → Cải tiến quy trình"

Ví dụ thực tế

Tình huống	TESTING (Khắc phục)	QA (Phòng ngừa)
App có nhiều bugs	Test và tìm tất cả bugs → Fix	Phân tích: Tại sao có nhiều bugs? → Thiếu code review → Thiết lập quy trình review bắt buộc
Security breach	Test lại security → Fix lỗ hổng	Rà soát quy trình development → Thêm security training → Security checklist bắt buộc
Performance issue	Load testing → Optimize code	Phân tích quy trình: → Thiếu performance testing sớm → Thêm performance test vào CI/CD

Trách nhiệm

```
┌──────────────────────────────────────────────────────────┐
│              AI CHỊU TRÁCH NHIỆM?                        │
```

```
│                                                                │
│                                                                 │
│  TESTING: Chủ yếu Testing Team                                 │
│  ├─ Testers                                                    │
│  ├─ QA Engineers                                               │
│  └─ Test Automation Engineers                                  │
│                                                                 │
│  QA: TRÁCH NHIỆM CỦA TẤT CẢ MỌI NGƯỜI                          │
│  ├─ Developers: Follow coding standards                        │
│  ├─ Testers: Follow testing process                            │
│  ├─ BA: Clear requirements                                     │
│  ├─ PM: Good project management                                │
│  └─ Everyone: Continuous improvement                           │
│                                                                 │
└────────────────────────────────────────────────────────────────┘
```

1.2.3. Error, Defect, Failure và Root Cause

Định nghĩa và mối quan hệ

```
CON NGƯỜI → ERROR → DEFECT → FAILURE

   👤          ❌         🐛         💥
 (Người)   (Sai lầm)  (Lỗi code) (Hệ thống fail)
```

Thuật ngữ	Tiếng Việt	Định nghĩa	Ví dụ
Error	Sai lầm, nhầm lẫn	Con người mắc lỗi	Developer hiểu sai yêu cầu, viết sai logic
Defect	Khuyết tật, lỗi	Sản phẩm có lỗi (do error gây ra)	Code: `if (x = 5)` thay vì `if (x == 5)`
Failure	Thất bại, lỗi hệ thống	Hệ thống không hoạt động đúng	App crash khi user login
Root Cause	Nguyên nhân gốc rễ	Lý do căn bản dẫn đến vấn đề	Thiếu training, áp lực deadline, quy trình kém

Quy trình từ Error đến Failure

```
┌──────────────────────────────────────────────────────────────┐
│          QUY TRÌNH TỪ SAI LẦM ĐẾN THẤT BẠI                    │
└──────────────────────────────────────────────────────────────┘


1. ROOT CAUSE (Nguyên nhân gốc)
   ├─ Áp lực deadline
   ├─ Thiếu training
   ├─ Quy trình phức tạp
   └─ Developer mệt mỏi
```

```
        │
        ↓
2. ERROR (Sai lầm của con người)
   └─ Developer hiểu sai requirements
        │
        ↓
3. DEFECT (Lỗi trong sản phẩm)
   ├─ Code sai logic
   ├─ Requirements sai
   ├─ Design sai
   └─ Test case sai
        │
        ↓ (nếu được thực thi)
        │
4. FAILURE (Hệ thống thất bại)
   └─ App crash, tính toán sai, security breach
```

Ví dụ cụ thể: Chuỗi Error → Defect → Failure

```
CASE STUDY: Bug tính lãi suất

ROOT CAUSE:
└─ Developer mới, chưa có kinh nghiệm về tài chính
   + Deadline gấp, không có thời gian review
        ↓
ERROR (Sai lầm):
└─ Developer hiểu sai công thức lãi suất
   (Nghĩ là lãi đơn, thực tế là lãi kép)
        ↓
DEFECT (Lỗi code):
└─ Code: interest = principal * rate * time
   (Sai! Nên là: interest = principal * (1 + rate)^time - principal)
        ↓
FAILURE (Thất bại):
└─ Khách hàng nhận được lãi suất SAI
   → Khiếu nại, mất tiền, mất uy tín
```

Giai đoạn	Diễn biến	Tác động
Root Cause	Dev mới + Deadline gấp + Không review	Môi trường dễ gây lỗi
Error	Hiểu sai công thức lãi kép	Không ai phát hiện (chưa)
Defect	Code công thức sai trong file `calculator.js`	Chưa gây hại (chưa chạy)
Failure	User tính lãi → Kết quả sai → Khiếu nại	Thiệt hại thực tế

Các loại Defects

Vị trí defect	Ví dụ	Hậu quả nếu không phát hiện
📄 **Tài liệu**	Requirements thiếu edge case	Code thiếu xử lý → Failure

💻 **Mã nguồn**	Logic sai, syntax error	App crash, tính toán sai
🏗️ **Build scripts**	Config file sai	Deploy fail, môi trường sai
🧪 **Test cases**	Test case sai	Không phát hiện được bug thực

Defect → Failure: Không phải lúc nào cũng xảy ra

```
┌──────────────────────────────────────────────────────┐
│      DEFECT có dẫn đến FAILURE không?                 │
├──────────────────────────────────────────────────────┤
│                                                       │
│  ✅ LUÔN DẪN ĐẾN FAILURE                              │
│     └─ Null pointer exception (luôn crash nếu chạy)   │
│                                                       │
│  ⚠️ CHỈ FAILURE TRONG ĐIỀU KIỆN CỤ THỂ                │
│     └─ Race condition (chỉ fail khi nhiều user cùng lúc)│
│                                                       │
│  ⭕ KHÔNG BAO GIỜ DẪN ĐẾN FAILURE                     │
│     └─ Dead code, unused variable (không ảnh hưởng)   │
│                                                       │
└──────────────────────────────────────────────────────┘
```

Nguyên nhân gây Failure

```
FAILURE không chỉ do DEFECT gây ra!

Nguyên nhân gây Failure:
├─ 🐛 Defects trong code (phổ biến nhất)
├─ ⚡ Điều kiện môi trường
│   ├─ Bức xạ
│   ├─ Trường điện từ
│   ├─ Nhiệt độ cực đoan
│   └─ Hardware failure
└─ 🔌 Infrastructure issues
    ├─ Network down
    ├─ Database crash
    └─ Server overload
```

Root Cause Analysis (Phân tích nguyên nhân gốc)

```
┌──────────────────────────────────────────────────────┐
│         ROOT CAUSE ANALYSIS                           │
└──────────────────────────────────────────────────────┘

Khi có FAILURE hoặc DEFECT:
    ↓
Thực hiện Root Cause Analysis
    ↓
Tìm ra NGUYÊN NHÂN GỐC RỄ
```

```
    ↓
Giải quyết nguyên nhân
    ↓
NGĂN CHẶN lỗi tương tự trong tương lai
```

Bước	Hoạt động	Ví dụ
1	Phát hiện Failure/Defect	50 bugs security trong sprint
2	Phân tích Root Cause	Tại sao có 50 bugs security?
3	Tìm nguyên nhân gốc	→ Thiếu security training → Không có security checklist → Code review không cover security
4	Giải quyết	→ Tổ chức security training → Tạo security checklist → Bổ sung security review
5	Kết quả	Sprint sau: chỉ còn 5 bugs security

Kỹ thuật Root Cause Analysis

```
5 WHYS (5 Tại sao)

Problem: App crash khi user login
│
├─ Why? → Code lỗi null pointer
│   │
│   ├─ Why? → Không validate input
│   │   │
│   │   ├─ Why? → Developer quên
│   │   │   │
│   │   │   ├─ Why? → Không có checklist
│   │   │   │   │
│   │   │   │   └─ Why? → Team không có quy trình review
│   │   │   │       │
│   │   │   │       └─ ROOT CAUSE: Thiếu quy trình đảm bảo chất lượng
```

Ví dụ thực tế: Root Cause Analysis

Vấn đề	Root Cause	Giải pháp	Kết quả
Nhiều UI bugs	Thiếu UI testing	Thêm visual regression testing	Giảm 80% UI bugs
Performance issues	Không có load testing	Thêm load test vào CI/CD	Phát hiện sớm bottleneck
Security breaches	Thiếu security awareness	Security training + SAST tools	Giảm 90% security bugs

Tóm tắt mối quan hệ

```
╔═══════════════════════════════════════════════════════╗
║        ERROR → DEFECT → FAILURE                       ║
╚═══════════════════════════════════════════════════════╝
║                                                       ║
║  ROOT CAUSE (Nguyên nhân gốc)                         ║
║        ↓                                              ║
║  ERROR (Con người sai lầm)                            ║
║        ↓                                              ║
║  DEFECT (Lỗi trong sản phẩm)                          ║
║        ↓ (nếu được thực thi)                          ║
║  FAILURE (Hệ thống không hoạt động đúng)              ║
║        ↓                                              ║
║  Root Cause Analysis                                  ║
║        ↓                                              ║
║  Giải quyết nguyên nhân gốc                           ║
║        ↓                                              ║
║  NGĂN CHẶN lỗi tương tự                               ║
║                                                       ║
╚═══════════════════════════════════════════════════════╝
```

Một số ghi chú

- Tài liệu dịch từ sách gốc ISTQB Foudation v4.0.1 sang tiếng Việt, nhằm mục đích giảm rào cản tiếp cận các kiến thức về testing tới cộng đồng tester Việt Nam nói chung và các anh chị em muốn tìm hiểu về testing nói riêng
- Tài liệu cố gắng dịch nhiều nhất các từ tiếng Việt để bạn đọc không phải tra từ điển trong quá trình đọc (VD: defects dịch là khuyết tật phần mềm).
- Dự án phi lợi nhuận, bạn có thể thoải mái sử dụng bản dịch, chia sẻ và sửa đổi nếu cần thiết.
- Có góp ý cho dự án, bạn có thể submit góp ý qua link này nha: https://go.betterbytesvn.com/sharing-documentation-feedback.
- Bản dịch được thực hiện bởi tác giả Đỗ Minh Phong. Bạn có thể gửi lời cảm ơn/feedback/ donate tới tác giả thông qua các hình thức:
  - Đăng ký kênh Youtube: https://www.youtube.com/@hoctest_com
  - Follow Fanpage: https://www.facebook.com/hoctest/
  - Tham gia group Playwright Việt Nam trên Facebook: https://www.facebook.com/groups/playwright.automation.test
  - Donate cho tác giả: https://academy.betterbytesvn.com/donate-cho-chung-minh/
  - Facebook cá nhân tác giả: https://www.facebook.com/dominhphong.18/

Xin chân thành cảm ơn bạn, vì đã quan tâm tới tài liệu ^^.